



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

APLICAȚIE MOBILĂ INTELIGENTĂ PENTRU
MANAGEMENT NUTRIȚIONAL ȘI
MONITORIZAREA PROGRESULUI
UTILIZATORULUI

Absolvent

Tunaru Ioana Alexandra

Coordonator științific

Lect. dr. Cezara Lamba-Benegui

București, iunie 2026

Rezumat

Lucrarea de față propune dezvoltarea BitePlan, o aplicație mobilă Android concepută pentru a îmbunătăți comunicarea dintre client și nutriționist. Aceasta oferă o alternativă personalizată față de aplicațiile de nutriție clasice bazate exclusiv pe autodisciplină. Platforma integrează funcționalități pentru ambele roluri: generarea și gestionarea planurilor nutriționale săptămânale, stabilirea obiectivului individual de greutate, planificarea consultațiilor și setarea limitelor zilnice de macronutrienți. Clientul are posibilitatea de a contribui activ la propriul regim alimentar prin sugestii de mese bazate pe produse scanate, decizia finală aparținând întotdeauna nutriționistului. Componentele inteligente includ detecția vizuală a alimentelor dintr-o fotografie și predicția evoluției greutății. Detecția este realizată cu modelul YOLOv8, care obține o performanță de 86,6% mAP50 pe subsetul de test, iar predicția utilizează modelul Random Forest Regressor care atinge un coeficient de determinare R^2 de 0,942. Arhitectura aplicației este organizată pe trei niveluri, reunind clientul Android (Kotlin și Jetpack Compose) cu serverul Node.js și microserviciul FastAPI Python, dedicat implementării componentelor inteligente. Datele sunt stocate în Azure SQL, iar integrarea API-urilor Spoonacular, Open Food Facts, DeepL și Groq contribuie la o soluție completă de nutriție personalizată.

Abstract

This paper proposes the development of BitePlan, an Android mobile application designed to improve communication between clients and nutritionists. It offers a personalized alternative to traditional nutrition applications based solely on self-discipline. The platform integrates functionalities for both roles: generating and managing weekly nutritional plans, setting individual weight goals, scheduling consultations and configuring daily macronutrient limits. The client has the opportunity to actively contribute to their own dietary regimen through meal suggestions based on scanned products, with the final decision always belonging to the nutritionist. The intelligent components include visual food detection from a photograph and prediction of weight progression. Detection is performed using the YOLOv8 model, achieving a performance of 86.6% mAP50 on the test set, while prediction utilizes the Random Forest Regressor model, obtaining a coefficient of determination R^2 of 0.942. The application architecture is organized into three layers, combining the Android client (Kotlin and Jetpack Compose) with the Node.js server and a FastAPI Python microservice dedicated to implementing the intelligent components. Data is stored in Azure SQL, and the integration of Spoonacular, Open Food Facts, DeepL and Groq APIs contributes to a complete personalized nutrition solution.

Cuprins

1	Introducere	7
1.1	Tipul lucrării și subdomeniul specific	7
1.2	Prezentarea generală a temei	7
1.3	Scopul și motivația alegerii temei	8
1.4	Contribuția proprie	8
1.5	Structura lucrării	8
2	Preliminarii	9
2.1	Noțiuni teoretice fundamentale	9
2.1.1	Arhitectura aplicației	9
2.1.2	Gestiunea datelor și persistența	9
2.1.3	Securitate și autentificare	9
2.1.4	Machine Learning	10
2.1.5	Computer Vision	10
2.2	Tehnologii utilizate	10
2.2.1	Dezvoltarea mobilă	10
2.2.2	Backend și servicii server	11
2.2.3	Gestiunea și stocarea datelor	11
2.2.4	API-uri externe	11
2.3	Soluții existente în domeniul nutriției	12
2.3.1	Aplicații similare	12
2.3.2	Studii despre aderența la planurile alimentare	12
2.3.3	Computer Vision în identificarea alimentelor	12
2.3.4	Fezabilitatea aplicației	13
2.4	Obiectivele lucrării	13
3	Contribuția propriu-zisă	14
3.1	Analiza problemei	14
3.1.1	Contextul problemei	14
3.1.2	Formularea clară a problemei	14
3.1.3	Descompunerea în subprobleme	15

3.1.3.1	Subprobleme tehnice	15
3.1.3.2	Subprobleme funcționale	15
3.1.3.3	Subprobleme algoritmice	16
3.2	Abordări existente	16
3.2.1	Aplicații reprezentative din domeniu	16
3.2.2	Detecția alimentelor prin Computer Vision	17
3.2.3	Limitări identificate și contribuția BitePlan	17
3.3	Implementarea soluției	18
3.3.1	Arhitectura aplicației	18
3.3.2	Predicția progresului prin Machine Learning	18
3.3.3	Recunoașterea alimentelor prin Computer Vision	19
3.3.4	Funcționalitatea de scanare a produselor	20
3.3.5	Construirea planurilor nutriționale	20
3.3.6	Probleme întâmpinate și soluții tehnice	21
3.4	Experimente și rezultate	21
3.4.1	Setul de date și configurația antrenării	21
3.4.1.1	Modelul Random Forest Regressor	21
3.4.1.2	Modelul YOLOv8	22
3.4.2	Metodologia de testare	23
3.4.3	Rezultate	24
3.4.3.1	Rezultatele modelului Random Forest Regressor	24
3.4.3.2	Rezultatele modelului YOLOv8	24
3.5	Analiza comparativă și interpretarea rezultatelor	24
3.5.1	Punctele forte ale soluției implementate	24
3.5.2	BitePlan în raport cu soluțiile actuale	25
3.5.3	Interpretarea personală a rezultatelor	25
3.5.4	Aplicabilitate reală și potențial de dezvoltare	25
4	Concluzii și propuneri viitoare	26
4.1	Concluzii generale	26
4.2	Perspective de dezvoltare	26
	Bibliografie	27
	Anexe	28
A	Implementarea arhitecturii MVVM și comunicarea cu backend-ul	29
B	Vectorul de caracteristici și fluxul de predicție	31
C	Motivarea alegerii hiperparametrilor pentru modelul YOLOv8	33

Capitolul 1

Introducere

1.1 Tipul lucrării și subdomeniul specific

Prezenta lucrare se încadrează în categoria „Aplicații software”, reprezentând dezvoltarea unei aplicații mobile pentru Android, concepută pentru a îmbunătăți colaborarea dintre client și nutriționist. Platforma oferă funcționalități dedicate ambelor tipuri de utilizatori, precum gestionarea planurilor alimentare și monitorizarea progresului.

În plus, integrarea serviciilor cloud, a API-urilor externe și a componentelor bazate pe inteligență artificială plasează lucrarea în domeniul „Tehnologii informatice și comunicaționale”.

1.2 Prezentarea generală a temei

Alimentația este un factor determinant al stării de sănătate care influențează direct calitatea vieții și longevitatea. În ciuda accesului tot mai larg la informații nutriționale, adoptarea unui comportament alimentar sănătos rămâne o provocare reală pentru majoritatea oamenilor.

Un studiu realizat pe 2.103 adulți din toate zonele României indică o prevalență de 31,1% pentru persoanele supraponderale și 21,3% pentru obezitate, obiceiurile alimentare nesănătoase identificate fiind mesele neregulate și consumul de alimente în fața televizorului [12].

Aceste rezultate sugerează că recomandările generice de tipul „one-size-fits-all” nu au în vedere nevoile diverse ale oamenilor, descurajând astfel consistența în respectarea lor. Studiile arată că planurile personalizate, bazate pe dieta curentă și caracteristicile măsurabile ale persoanei, precum fenotipul sau informațiile genetice, determină rezultate semnificativ mai bune [7]. BitePlan răspunde acestei necesități prin integrarea nutriționistului direct în aplicație și favorizarea comunicării continue între specialist și utilizatorii interesați să își îmbunătățească stilul de viață.

1.3 Scopul și motivația alegerii temei

Principalul scop al aplicației BitePlan este să transforme relația dintre client și nutriționist într-un proces mai accesibil și interactiv, în care utilizatorul poate contribui activ prin transmiterea feedback-ului și exprimarea preferințelor personale.

Motivația implementării acestei soluții provine din experiențe personale cu dificultăți în respectarea unor indicații alimentare și stigmatizarea dorinței de a apela la un specialist. BitePlan a fost construită pentru persoanele care ar apela la ajutor profesional dacă ar avea la îndemână o aplicație care să îi însoțească pe tot parcursul procesului de educare nutrițională și de formare a alegerilor alimentare conștiente.

1.4 Contribuția proprie

Contribuția proprie constă în dezvoltarea aplicației mobile BitePlan care urmărește promovarea unei relații mai bune client-nutriționist, dar și client-alimentație. Aplicația propune o abordare unică prin introducerea unor funcționalități care îi permit clientului să contribuie direct la propriul regim alimentar.

Printre elementele distinctive ale aplicației se numără dezvoltarea unui mecanism de sugerare a meselor bazat pe codul de bare al produsului favorit, adaptat obiectivelor nutriționale individuale și integrat în procesul de validare realizat de nutriționist. De asemenea, asistarea procesului de organizare a cumpărăturilor prin detecția inteligentă a alimentelor dintr-o fotografie, precum și predicția numărului de săptămâni până la atingerea greutății dorite, bazată pe aderența la planul alimentar, contribuie la menținerea motivației clientului pe termen lung.

Prin aceste componente, BitePlan se diferențiază de aplicațiile similare de nutriție, urmărind să îndeplinească atât nevoile clientului privind ghidarea personalizată, cât și nevoia specialiștilor de a eficientiza procesul de introducere manuală a datelor prin funcționalități inteligente [8] [1].

1.5 Structura lucrării

Restul lucrării este structurat după cum urmează. Capitolul 2 prezintă noțiunile teoretice fundamentale care stau la baza aplicației, anume arhitectura sistemului, gestiunea datelor, securitatea, Machine Learning și Computer Vision, tehnologiile utilizate, soluțiile existente și obiectivele lucrării.

Capitolul 3 constituie contribuția propriu-zisă și cuprinde analiza problemei, implementarea soluției BitePlan și evaluarea funcționalităților inteligente dezvoltate.

Capitolul 4 sintetizează concluziile lucrării și evidențiază direcțiile de dezvoltare ulterioară a aplicației.

Capitolul 2

Preliminarii

2.1 Noțiuni teoretice fundamentale

2.1.1 Arhitectura aplicației

Platforma adoptă modelul arhitectural client-server, iar la nivelul aplicației mobile este utilizată arhitectura MVVM (Model-View-ViewModel), care separă logica de business de interfața grafică. Comunicarea cu serverul principal Node.js se realizează prin intermediul bibliotecii Retrofit, un client HTTP type-safe pentru Android, care gestionează automat serializarea și deserializarea datelor în format JSON. În plus, aplicația utilizează principiul Dependency Injection, prin care obiectele nu își creează singure dependențele, ci le primesc din exterior.

Complexitatea funcționalităților inteligente ale aplicației a impus extinderea structurii clasice printr-un al treilea nivel, anume microserviciul Python dedicat exclusiv modelelor de Machine Learning și Computer Vision.

2.1.2 Gestiunea datelor și persistența

Datele aplicației sunt stocate într-o bază de date relațională găzduită în cloud, care asigură integritatea datelor prin chei externe și constrângeri SQL. Fișierele binare, precum fotografiile de profil, sunt stocate în Firebase Storage, în baza de date fiind reținut doar URL-ul asociat acestora.

Comunicarea dintre Node.js și baza de date se realizează prin intermediul unui driver dedicat. Pentru optimizarea performanței, serverul utilizează un *connection pool* în vederea gestionării eficiente a conexiunilor.

2.1.3 Securitate și autentificare

Controlul accesului în aplicație este gestionat de Firebase Authentication, un serviciu care implementează fluxuri standardizate bazate pe token-uri. Platforma adoptă atât au-

tentificare clasică prin e-mail și parolă, cât și prin contul Google. În cazul autentificării cu succes, Firebase emite un token JWT (JSON Web Token) care înglobează identitatea utilizatorului. Tokenul este transmis în headerul fiecărei cereri HTTP către serverul Node.js, unde un middleware de autentificare îl interceptează și îi verifică validitatea înainte ca aceasta să ajungă la logica de business.

Același mecanism de autentificare se extinde și în comunicarea cu microserviciul Python, prin respectarea principiului token forwarding. Tokenul primit de la clientul Android este retransmis către serverul Node.js, care rămâne singurul responsabil de validarea identității și accesul la date.

2.1.4 Machine Learning

Funcționalitatea de predicție a progresului se bazează pe Random Forest Regressor, un algoritm de învățare automată supervizată de tip ansamblu, care construiește un număr mare de arbori de decizie pe subseturi aleatorii ale datelor și combină predicțiile prin medie.

Modelul primește ca intrare un vector de caracteristici, care combină date ale profilului utilizatorului (gen, vârstă, înălțime, greutate curentă și greutate dorită, obiectiv caloric și nivel de activitate) cu date dinamice extrase din comportamentul recent: tendința greutății, aderența la planul alimentar, deficitul caloric și cheltuiala energetică totală zilnică.

2.1.5 Computer Vision

Funcționalitatea de detectare automată a alimentelor se bazează pe tehnici de Computer Vision, domeniu care vizează extragerea informațiilor semantice din imagini digitale. Detecția de obiecte identifică și localizează simultan multiple elemente într-o imagine, generând pentru fiecare o etichetă și o zonă de delimitare (bounding box).

Modelul utilizat este YOLOv8, un detector care procesează întreaga imagine o singură dată printr-o rețea neuronală convoluțională. Această abordare one-stage îi conferă un raport superior între viteză și acuratețe față de arhitecturile two-stage, care generează mai întâi propuneri de regiuni și le clasifică într-un pas separat.

2.2 Tehnologii utilizate

2.2.1 Dezvoltarea mobilă

BitePlan este o aplicație Android nativă, dezvoltată în limbajul Kotlin. Față de soluțiile cross-platform precum Flutter, abordarea nativă oferă acces direct la API-urile platformei, esențial pentru integrarea cu ML Kit și accesul la camera dispozitivului, fără

niveluri suplimentare de abstractizare. Pentru gestionarea dependențelor a fost ales Koin, potrivit ca dimensiune și configurare pentru complexitatea aplicației.

Interfața grafică este construită cu Jetpack Compose, care permite definirea UI-ului prin funcții composable și gestionează automat recompunerea interfeței la schimbarea stării.

2.2.2 Backend și servicii server

Serverul aplicației este implementat în Node.js, ales pentru eficiența sa în operații de tip I/O precum cereri HTTP, acces la baza de date și integrarea cu servicii externe. Această caracteristică îl face potrivit pentru BitePlan, unde comunicarea frecventă cu API-uri externe și procesarea cererilor simultane sunt esențiale.

Pentru organizarea backend-ului a fost utilizat Express.js, care permite structurarea clară a endpoint-urilor în rute, controllere și servicii. În cazul BitePlan, această separare este utilă pentru gestionarea independentă a funcționalităților precum autentificare, planuri alimentare, notificări și integrări cu API-uri externe.

2.2.3 Gestiunea și stocarea datelor

Datele aplicației sunt stocate în Azure SQL Database, ales pentru structura relațională a conținutului. Baza de date conține entități precum utilizatori, profiluri, planuri alimentare, programări, obiective, între care există relații clare, precum asocierea dintre nutriționiști și clienții lor.

Interogarea bazei de date se realizează prin T-SQL (Transact-SQL), dialectul Microsoft al limbajului SQL, care extinde sintaxa standard cu construcții procedurale precum variabile, structuri condiționale și gestionarea tranzacțiilor.

2.2.4 API-uri externe

Aplicația integrează patru servicii externe prin apeluri HTTP: Spoonacular API, Open Food Facts API, DeepL API și Groq API.

Spoonacular API este cel mai frecvent utilizat: prin intermediul său nutriționistul construiește planurile alimentare, clienții primesc sugestii de mese bazate pe un produs scanat și aplicația generează liste de cumpărături cu ingredientele necesare. Open Food Facts API este utilizat în componenta de scanare a codului de bare, returnând informații nutriționale despre produs.

DeepL API este utilizat pentru traducerea automată în engleză a listei de ingrediente pentru produsele scanate, asigurând uniformitatea datelor trimise către celelalte servicii. Groq API oferă acces la LLM-uri, fiind folosit pentru patru sarcini: extragerea ingredientului principal dintr-un nume de produs, generarea de descrieri scurte pentru rețete,

formatarea listelor de ingrediente și validarea etichetelor introduse de client în mecanismul de fine-tuning.

2.3 Soluții existente în domeniul nutriției

2.3.1 Aplicații similare

MyFitnessPal este una dintre cele mai utilizate aplicații de monitorizare nutrițională, utilizatorii având posibilitatea să își înregistreze aportul caloric, greutatea și activitatea fizică. Platforma se concentrează exclusiv pe monitorizare individuală, fără a oferi ghidare personalizată în procesul de stabilire și urmărire a obiectivelor.

Noom este o aplicație destinată atingerii obiectivelor de greutate care, spre deosebire de platformele clasice de monitorizare calorică, adoptă o abordare psihologică orientată spre schimbarea comportamentului alimentar. Aceasta oferă utilizatorilor conținut educațional zilnic despre modificarea obiceiurilor alimentare și acces la un specialist prin mesagerie în aplicație, al cărui rol este de a oferi suport motivațional și comportamental, nu ghidare nutrițională specializată.

2.3.2 Studii despre aderența la planurile alimentare

Aderența pe termen lung la planurile alimentare reprezintă una dintre principalele provocări în domeniul aplicațiilor de nutriție. Un studiu retrospectiv pe 19.805 utilizatori ai unei aplicații publice arată că doar 8,5% rămân consecvenți pe termen lung, factorii asociați cu o mai bună aderență fiind utilizarea tutorialelor, setarea reminderelor și personalizarea conținutului [6].

Totodată, efortul necesar pentru monitorizarea alimentară este considerat unul dintre principalii factori care contribuie la abandonarea utilizării aplicațiilor de nutriție [5]. Impactul acestei probleme este evidențiat și într-un studiu desfășurat pe 124 de participanți, care arată că mai puțin de jumătate dintre utilizatori au continuat să își înregistreze datele alimentare după săptămâna a zecea [14].

2.3.3 Computer Vision în identificarea alimentelor

Identificarea automată a alimentelor din imagini reprezintă un domeniu activ de cercetare în Computer Vision, cu aplicabilitate directă în monitorizarea dietei.

Primele abordări bazate pe extragere manuală a caracteristicilor vizuale au obținut rezultate limitate, însă introducerea rețelelor neuronale convoluționale a îmbunătățit semnificativ acuratețea recunoașterii alimentelor [10], deschizând calea spre aplicații mobile capabile să identifice automat conținutul unei mese dintr-o fotografie.

Modelele de tip YOLO reprezintă o evoluție importantă în acest domeniu, trecând de la clasificarea unei singure etichete per imagine la detectarea simultană a mai multor alimente, fiecare localizat printr-un *bounding box*. Acestea fac parte din categoria detectoarelor *one-stage*, care procesează imaginea o singură dată [13], ceea ce le permite să funcționeze în timp real chiar și pe dispozitive mobile cu resurse limitate și să identifice mai multe alimente dintr-o singură fotografie realizată de utilizator.

2.3.4 Fezabilitatea aplicației

Cercetările din domeniu evidențiază limitările clare ale aplicațiilor de nutriție existente din perspectiva specialiștilor. Un studiu pe 381 de dieteticieni arată că aplicațiile actuale sunt dificil de utilizat din cauza introducerii manuale a datelor și că aceștia preferă să eficientizeze procesul de evaluare alimentară [1].

Implicarea unui specialist în nutriție în cadrul unei aplicații mobile are un impact măsurabil asupra comportamentului utilizatorului. Un experiment randomizat pe patru luni demonstrează că suportul unui dietetician crește angajamentul utilizatorului cu 50,7%, facilitând tranziția de la intenție la comportament real [8]. Aceste rezultate confirmă fezabilitatea platformei BitePlan, în care nutriționistul supervizează planul alimentar, iar componentele inteligente precum scanarea codului de bare, recunoașterea AI a alimentelor și predicția ML vin în completarea acestuia.

2.4 Obiectivele lucrării

Obiectivul principal al acestei lucrări este îmbunătățirea comunicării dintre client și nutriționist prin intermediul unui sistem digital structurat, care să promoveze un stil de viață echilibrat. Motivația este susținută de cercetări care demonstrează dificultatea menținerii autodisciplinii alimentare pe termen lung și importanța implicării unui specialist.

Din perspectivă tehnică, lucrarea urmărește integrarea inteligenței artificiale într-o aplicație reală de nutriție și evaluarea beneficiilor acesteia în experiența utilizatorului. Componentele inteligente vizează eficientizarea introducerii datelor prin scanarea codului de bare, recunoașterea automată a alimentelor, precum și oferirea unei predicții privind atingerea obiectivului de greutate pentru menținerea motivației utilizatorului.

Din punct de vedere al nutriției, se urmărește demonstrarea fezabilității respectării unui plan alimentar în contextul în care nutriționistul primește feedback activ din partea clientului, iar clientul poate contribui cu sugestii de mese.

Prin combinarea supervizării specializate cu tehnologii de inteligență artificială, BitePlan își propune să răspundă unor limitări identificate în aplicațiile existente și să ofere o soluție viabilă pentru gestionarea unui plan personalizat.

Capitolul 3

Contribuția propriu-zisă

3.1 Analiza problemei

3.1.1 Contextul problemei

Persoanele care își doresc să mențină un stil de viață sănătos se confruntă zilnic cu alegeri alimentare dificile din cauza volumului mare de informații contradictorii din mass-media și a tendințelor culturale legate de imaginea corporală. Un studiu realizat pe un eșantion de 1.185 participanți români cu vârsta de peste 18 ani evidențiază că suprasaturarea informațională generează confuzie și conduce la renunțarea treptată la un comportament alimentar echilibrat [15]. BitePlan oferă utilizatorilor o platformă unde informațiile sunt afișate într-un mod organizat și adaptat nevoilor individuale.

Majoritatea aplicațiilor existente oferă planuri alimentare generice, fără o supervizare specializată și fără comunicare bidirecțională între client și nutriționist. În acest sens, simpla afișare a valorilor nutriționale sau logarea într-un jurnal alimentar nu mai reprezintă o soluție suficientă pentru utilizatorii care urmăresc rezultate reale și de durată.

3.1.2 Formularea clară a problemei

Aplicațiile de nutriție au redus încrederea utilizatorilor deoarece promit rezultate rapide și progres vizibil într-un timp limitat, fără a fi susținute de informații corecte și recomandări personalizate. Un studiu realizat în 2026 care urmărește intenția de utilizare a aplicațiilor de dietă demonstrează că utilizatorii care percep platformele ca fiind construite cu implicarea unui specialist manifestă o dorință semnificativ mai mare de utilizare, influențată de percepția utilității acesteia [3]. În plus, specialiștii își exprimă îngrijorarea că platformele nutriționale existente le ignoră expertiza, aceștia dorind ca aplicațiile să îmbunătățească relația cu pacienții și să faciliteze o colaborare fluidă [1].

Dincolo de contextul social, construirea unei aplicații care să adreseze în mod real aceste lipsuri presupune provocări tehnice considerabile. O primă dificultate este repre-

zentată de gestionarea unui flux de date eterogen: extragerea de informații nutriționale din surse diverse, precum Spoonacular API, respectiv Open Food Facts API și integrarea acestora în funcționalități folositoare utilizatorului. A doua provocare vizează personalizarea reală a planurilor alimentare, care urmărește atât feedback-ul subiectiv al clientului, cât și evoluția greutății prin modele de Machine Learning. În plus, utilizarea tehnologiilor de Computer Vision pentru simplificarea gestionării manuale a listei de cumpărături introduce o dificultate tehnică suplimentară, concretizată în nevoia unui proces continuu de fine-tuning al modelului, pentru a garanta o acuratețe superioară.

În acest sens, BitePlan urmărește crearea unui spațiu sigur atât pentru clienți, cât și pentru nutriționiști, în care există posibilitatea unei ghidări eficiente pe parcursul întregului proces de schimbare a comportamentului alimentar.

3.1.3 Descompunerea în subprobleme

3.1.3.1 Subprobleme tehnice

Securitate și infrastructură: Platforma urmărește o arhitectură pe trei niveluri, Android - Node.js - FastAPI Python, unde autentificarea este gestionată prin token-uri Firebase, validate la nivelul serviciilor backend. Separarea asigură că microserviciul Python poate solicita date despre client de la backend-ul principal doar în cadrul unei sesiuni autentificate.

Interfața mobilă: Menținerea coerenței datelor afișate indiferent de ciclul de viață al ecranului este dificil de garantat deoarece ecranele aplicației reunesc surse multiple asincrone de date, anume baza de date, API-uri externe și microserviciul Python. Soluția adoptată combină ViewModel-ul, care deține starea și supraviețuiește recompoziției ecranului, cu StateFlow care propagă modificările de date în timp real către interfață.

Preferințe locale: Preferința utilizatorului de temă vizuală (light/dark) reprezintă o setare specifică dispozitivului, nu contului utilizatorului, ceea ce a exclus stocarea ei în baza de date relațională. Această funcționalitate a fost soluționată cu ajutorul unui mecanism de stocare locală, anume Android DataStore care, datorită operațiilor asincrone de citire și scriere rulate pe un fir de execuție secundar, nu blochează interfața grafică a aplicației.

3.1.3.2 Subprobleme funcționale

Sistemul de notificări: Rutarea notificărilor către ecranul corect a necesitat diferențierea în funcție de rolul utilizatorului, client sau nutriționist, și identificarea entității referențiate prin intermediul unui câmp flexibil de metadata care stochează contextul necesar navigării post-click.

Sistemul de programări: Logica de rezervare a presupus calculul dinamic al sloturilor disponibile în funcție de zilele de lucru ale nutriționistului, intervalele orare stabilite de

acesta și durata variabilă a consultațiilor. În plus, statusul programărilor este gestionat automat la pornirea serverului, care marchează drept „Completed” toate programările confirmate sau în așteptare a căror oră de final se află în trecut.

3.1.3.3 Subprobleme algoritmice

Predicția progresului: Funcționalitatea de predicție utilizează Random Forest Regressor antrenat pe un vector de caracteristici construit din date eterogene. Dificultatea în implementare a constituit-o găsirea unui set de date real adecvat nevoilor componentei. Soluția adoptată a fost generarea unui set de date sintetic, simulând evoluția greutateii unor clienți pe o perioadă de șase luni pe baza unor fluctuații de aderență realiste.

Detectarea alimentelor din imagini: Recunoașterea automată a alimentelor într-o fotografie reprezintă o subproblemă algoritmică complexă, datorită instabilității condițiilor de iluminare și suprapunerii obiectelor. Această componentă adoptă modelul YOLOv8 care obține valoarea mAP de 86,6% pe subsetul de testare. Pentru a menține componenta de Computer Vision cât mai precisă, s-a introdus un mecanism de fine-tuning bazat pe corecțiile utilizatorilor asupra etichetelor bounding box-urilor prezise de model.

Procesarea și normalizarea datelor alimentare: Componenta de scanare a codului de bare generează un nume de produs incompatibil cu interogările către Spoonacular API. Soluția constă în utilizarea Groq API, care extrage ingredientul reprezentativ din numele produsului pe baza unui prompt structurat ce instruește modelul LLM să înțeleagă contextul denumirii și să returneze termenul cel mai relevant.

3.2 Abordări existente

3.2.1 Aplicații reprezentative din domeniu

Aplicațiile prezentate urmăresc ca utilizatorul să își monitorizeze singur regimul alimentar și să fie autodidact în ceea ce privește alegerea cantităților de macronutrienți potrivite și a tipurilor de alimente pe care să le includă. MyFitnessPal și Noom sunt două dintre cele mai utilizate aplicații de management al greutății, fiecare cu o abordare distinctă.

MyFitnessPal pune accentul pe monitorizarea calorică și a macronutrienților, însă un studiu realizat pe 1.4 milioane de utilizatori relevă că doar 18,2% dintre obiectivele de slăbire sunt atinse [4], evidențiind o limitare fundamentală a aplicației: absența unei ghidări personalizate în procesul de atingere a obiectivelor.

Pentru a evalua eficiența aplicațiilor mHealth în combaterea obezității, un studiu pe 35.921 de utilizatori ai platformei Noom a arătat că 77,9% au înregistrat o scădere în greutate, însă factorul cel mai important pentru succes a fost frecvența cu care utilizatorii își înregistrau cina [2]. Acest rezultat sugerează că eficiența aplicațiilor de nutriție rămâne

dependentă de autodisciplina și consecvența utilizatorului, în absența unei supervizări specializate.

3.2.2 Detecția alimentelor prin Computer Vision

NutriNet este o arhitectură de rețea neuronală convoluțională dedicată recunoașterii alimentelor și băuturilor pe un dataset de 225.953 de imagini repartizate în 520 de clase. Aceasta obține o acuratețe de 86,72% pe setul de recunoaștere și 94,47% pe setul de detecție, fiind integrată într-o aplicație mobilă reală pentru evaluarea dietei pacienților cu Parkinson [10]. Un element relevant este mecanismul de online training: modelul se îmbunătățește automat cu imagini noi primite de la utilizatori, similar principiului de fine-tuning continuu implementat în BitePlan.

FoodTracker este o aplicație mobilă care detectează simultan multiple alimente dintr-o fotografie în timp real, folosind un model one-stage bazat pe YOLOv2, obținând o valoare mAP de 76,36% cu un timp de inferență de 75 ms per imagine [13]. După detecție, aplicația returnează automat informațiile nutriționale ale alimentelor identificate. O limitare relevantă este că modelul a fost antrenat predominant pe alimente asiatice, reducând aplicabilitatea pentru utilizatorii din alte regiuni. De asemenea, platforma nu integrează un plan alimentar personalizat sau supervizarea unui specialist.

3.2.3 Limitări identificate și contribuția BitePlan

Analiza aplicațiilor existente evidențiază o limitare comună: utilizatorii abandonează frecvent înainte de a produce o schimbare reală în comportamentul alimentar, principala cauză fiind timpul ridicat investit în utilizarea aplicației [5]. Funcționalități precum scanarea codului de bare și recunoașterea vizuală a alimentelor reprezintă soluții pentru reducerea acestui efort, ambele fiind integrate în BitePlan.

Absența suportului specializat reprezintă o a doua limitare semnificativă. Un studiu randomizat controlat pe utilizatori Noom arată că grupul care a beneficiat de intervenție zilnică din partea unui terapeut a înregistrat o pierdere în greutate semnificativ mai mare față de grupul care a folosit aplicația independent (-3,1% vs -0,7%), confirmând că intervențiile digitale sunt cele mai eficiente când includ suport uman [9].

BitePlan adresează ambele limitări identificate prin mecanisme automate de identificare a alimentelor care elimină introducerea manuală, prin implicarea unui nutriționist licențiat care supervizează și ajustează planul alimentar și prin predicția ML a progresului pentru menținerea motivației utilizatorului pe termen lung.

3.3 Implementarea soluției

3.3.1 Arhitectura aplicației

Platforma BitePlan implementează o arhitectură distribuită pe trei niveluri, combinând clientul Android cu serverul Node.js și microserviciul Python, iar accesul la baza de date este realizat printr-o conexiune directă la nivelul backend-ului.

Primul nivel al sistemului este reprezentat de aplicația mobilă Android, responsabilă de asigurarea interacțiunii directe dintre utilizator și funcționalitatea platformei. Framework-ul declarativ Jetpack Compose este utilizat pentru interfața grafică, logica aplicației urmează arhitectura MVVM, iar pentru implementarea mecanismului de Dependency Injection este utilizată biblioteca Koin.

Al doilea nivel cuprinde backend-ul principal, implementat în Node.js cu Express.js, organizat modular pentru separarea clară a rutării, logicii de business, integrării cu servicii externe și accesului la baza de date. Fiecare endpoint este protejat printr-un middleware de autentificare care verifică validitatea token-ului Firebase înainte de procesarea cererii. Azure SQL Database stochează entitățile aplicației, iar arhitectura MVVM și comunicarea cu servicii externe sunt ilustrate în Anexa A, Figura A.1.

Al treilea nivel este microserviciul Python, implementat cu FastAPI și separat deliberat de backend-ul principal pentru a izola componentele inteligente de restul platformei.

3.3.2 Predicția progresului prin Machine Learning

Fluxul începe în momentul în care aplicația mobilă transmite token-ul de autentificare către microserviciul Python, care solicită backend-ului Node.js trei categorii de informații: profilul clientului, istoricul înregistrărilor de greutate și aderența zilnică la planul alimentar, calculată ca medie ponderată a statusurilor meselor: masă consumată integral (scor 1,0), consumată parțial (scoruri posibile: 0,25, 0,50, 0,75) sau sărită (scor 0,0).

Predicția este validă doar dacă utilizatorul a înregistrat cel puțin patru măsurători de greutate în ultimele 30 de zile, prag necesar pentru estimarea unui trend statistic valid.

Un element central în această funcționalitate îl reprezintă calculul necesar energetic zilnic (TDEE) al clientului pentru a determina dacă acesta se află în deficit caloric sau în surplus caloric. Calculul pornește de la rata metabolică bazală (BMR), care reprezintă cantitatea de energie consumată de organism în repaus complet, estimată prin formula Mifflin-St Jeor, diferențiată pe gen [11]:

$$\text{BMR}_{\text{bărbat}} = 10w + 6.25h - 5a + 5 \quad (3.1)$$

$$\text{BMR}_{\text{femeie}} = 10w + 6.25h - 5a - 161 \quad (3.2)$$

unde w reprezintă greutatea în kilograme, h înălțimea în centimetri și a vârsta în ani.

Necesarul energetic zilnic total (TDEE) se obține apoi prin înmulțirea BMR-ului cu un factor de activitate fizică:

$$\text{TDEE} = \text{BMR} \times \text{factor}_{\text{activitate}} \quad (3.3)$$

căruia i se atribuie următoarele valori utilizate în practica nutrițională clinică: sedentar (1,20), ușor activ (1,375), activ (1,55) și foarte activ (1,725).

Pe baza datelor agregate se construiește un vector de 15 caracteristici organizate în patru categorii: greutate, aderență, aport caloric și profil, detaliate în Anexa B.

Modelul utilizat este *Random Forest Regressor*, configurat cu 100 de arbori de decizie și integrat într-un pipeline care include o etapă de normalizare a caracteristicilor prin *StandardScaler*, necesară pentru a uniformiza măsurătorile diverse ale vectorului.

Predicția efectivă estimează ritmul săptămânal de modificare a greutății și calculează numărul de săptămâni până la atingerea obiectivului țintă, limitat la maximum 52 de săptămâni și invalidat când rata scade sub 0,05 kg pe săptămână. Răspunsul transmis aplicației Android include numărul estimat de săptămâni, rata săptămânală, un interval de încredere orientativ și o serie de puncte pentru afișarea trendului grafic. Fluxul complet al predicției este ilustrat în Anexa B, Figura B.1.

3.3.3 Recunoașterea alimentelor prin Computer Vision

Listele de cumpărături generate pe baza planurilor alimentare devin greu de monitorizat prin verificare manuală. Funcționalitatea răspunde acestei nevoi prin organizarea rapidă a listei de cumpărături, identificând alimentele sau ingredientele dintr-o fotografie.

Clientul poate selecta o imagine din galerie sau fotografia direct cumpărăturile, iar aceasta este transmisă către microserviciul Python prin biblioteca Retrofit. Modelul YOLOv8, încărcat la pornirea microserviciului, procesează imaginea și returnează coordonatele bounding box-urilor, eticheta și scorul de încredere pentru alimentele detectate. Alimentele din lista inițială de cumpărături care se regăsesc printre cele detectate sunt încadrate drept „Already have”, iar celelalte rămân cu statusul „Still missing”.

Clienții sunt îndemnați să corecteze etichete greșite cu numele alimentului corect în limba engleză. Înainte de stocare, corecțiile sunt normalizate, anume convertite la litere mici și la forma de singular prin pachetul *pluralize*. Groq API verifică dacă eticheta introdusă este un ingredient alimentar, respingând valori precum „nothing” sau denumiri de obiecte. Astfel, în baza de date ajung exclusiv adnotări relevante pentru fine-tuning.

Mecanismul de fine-tuning pornește de la un script Python care extrage periodic adnotările din baza de date și le convertește în format YOLO, normalizând coordonatele bounding box-urilor în intervalul [0,1] și împarte setul de date în 80% antrenare și 20% validare. Acesta este încărcat în Google Colab unde modelul existent este reantrenat pentru 20 de epoci, cu primele 10 straturi înghețate (`freeze=10`) pentru a păstra reprezentările

generale învățate inițial și a adapta doar straturile superioare pe noile date. Dacă acuratețea modelului reantrenat este îmbunătățită față de cel curent, acesta îl înlocuiește în microserviciu. Fine-tuning-ul este limitat de lipsa automatizării, rularea scriptului și reantrenarea se execută manual după colecționarea mai multor adnotări.

3.3.4 Funcționalitatea de scanare a produselor

Această funcționalitate îndeamnă clientul să participe activ la construirea propriului regim alimentar. Fluxul începe prin scanarea codului de bare utilizând Google ML Kit, iar pe baza codului rezultat, Open Food Facts API returnează informații nutriționale despre produs (macronutrienți, ingrediente, numele, producătorul și scorul nutrițional). Ingredientele sunt traduse automat în limba engleză prin DeepL API pentru a uniformiza ecranul.

Pe baza produsului scanat, clientul poate solicita sugestii de mese pentru un anumit moment al zilei, respectiv mic dejun, prânz, cină sau gustare. Sugestiile sunt filtrate să se încadreze în limitele impuse de specialist, distribuite proporțional per tip de masă în concordanță cu recomandările clinice nutriționale: mic dejun (25%), prânz (35%), cină (30%), gustare (10%).

Sugestiile de mese sunt generate prin Spoonacular API, care primește ca parametri ingredientul principal al produsului scanat, tipul mesei și numărul maxim de macronutrienți. Ingredientul principal este extras din numele produsului scanat folosind Groq API, care procesează textul și returnează termenul cel mai relevant din interogare. De asemenea, acest serviciu este utilizat pentru formatarea listei de ingrediente și generarea descrierilor scurte ale meselor sugerate.

Clientul poate vizualiza detalii despre fiecare sugestie, precum descriere, macronutrienți, ingrediente, și poate trimite o cerere către nutriționist însoțită opțional de o notă personală. Specialistul primește cererea printr-o notificare și decide dacă masa sugerată este adăugată sau nu într-un viitor plan alimentar. Astfel, clientul are posibilitatea să participe la personalizarea regimului său, dar decizia finală rămâne în responsabilitatea nutriționistului.

3.3.5 Construirea planurilor nutriționale

Nutriționistul este responsabil de crearea unui plan personalizat pentru clienții săi, bazat pe feedback-ul primit și pe obiectivele stabilite. În ecranul dedicat planurilor alimentare, valorile maxime zilnice sunt afișate permanent, facilitând respectarea limitelor la adăugarea meselor.

Mesele sunt căutate folosind Spoonacular API, care acceptă ca parametri cuvântul cheie introdus de nutriționist (de exemplu „pasta”), precum și filtre opționale: tipul dietei (vegetarian, vegan, ketogenic), intoleranțe alimentare (gluten, arahide), timpul maxim

de preparare (10-120 min) și intervalul caloric (100-800 kcal). Aceste filtre eficientizează procesul de construire a planului nutrițional, rezolvând nevoia specialiștilor ca funcționalitățile să fie automatizate cu o introducere manuală minimă. Căutarea filtrată va afișa până la 20 de mese, oferind suficiente opțiuni pentru crearea unui plan divers și echilibrat.

Fiecare zi din plan acceptă câte o singură masă per tip: mic dejun, prânz, cină și gustare, iar nutriționistul este atenționat vizual în cazul depășirii limitelor de macronutrienți. Întreaga săptămână poate fi duplicată până la 12 săptămâni în avans, eliminând un efort repetitiv al specialistului. Clientul are acces la planul alimentar exclusiv în momentul publicării, până atunci acesta rămâne în starea de „Draft”.

3.3.6 Probleme întâmpinate și soluții tehnice

Provocările legate de funcționalitățile inteligente au vizat lipsa seturilor de date adecvate. Pentru predicția progresului s-a adoptat ca soluție generarea unui set de date sintetic, care simulează un comportament alimentar realist, pe baza unor formule logice validate. În cazul detecției alimentelor, niciun set de date nu acoperea suficiente clase, soluția fiind concatenarea manuală a mai multor seturi astfel încât să rezulte 45 de tipuri de alimente. Pentru anumite categorii, concatenarea a fost necesară inclusiv la nivel individual, un singur set de date nefiind suficient pentru a asigura un volum adecvat de imagini.

Filtrarea corecțiilor utilizatorilor a impus probleme în realizarea mecanismului de fine-tuning. Dacă în setul de adnotări ajungeau corecții care nu reprezentau produse alimentare, calitatea datelor de antrenare ar fi fost compromisă. Soluția a constat în integrarea Groq API care a stabilit dacă eticheta propusă este un ingredient alimentar valid, respingând automat orice corecție irelevantă înainte de stocare.

În cadrul funcționalității de scanare a codului de bare, o dificultate a apărut în compatibilitatea dintre denumirile comerciale returnate de Open Food Facts API și parametrii acceptați de Spoonacular API pentru căutarea de rețete. Groq API a rezolvat această problemă prin extragerea ingredientului reprezentativ din denumirea produsului, permițând continuarea funcționalității.

3.4 Experimente și rezultate

3.4.1 Setul de date și configurația antrenării

3.4.1.1 Modelul Random Forest Regressor

Setul de date a reprezentat o provocare deoarece informațiile precum istoricul greutăților au caracter sensibil și nu sunt disponibile public, alternativa constând în generarea

unui set de date sintetic. Acesta a fost realizat pe baza a 100 de clienți simulați pe o perioadă de șase luni, cu 6-10 măsurători pe lună.

Utilizatorii au fost generați pornind de la 22 de profiluri de bază, acoperind scenarii de slăbire ușoară (2-5 kg), moderată (5-15 kg) și semnificativă (15+ kg), respectiv îngrășare ușoară (3-7 kg) și moderată (7+ kg). Pentru a asigura diversitatea datelor și o generalizare optimă a modelului, fiecare profil de bază a fost perturbat aleatoriu: greutatea inițială cu ± 3 kg, greutatea țintă cu ± 2 kg, înălțimea cu ± 2 cm, vârsta cu ± 3 ani și obiectivul caloric cu ± 100 kcal, rezultând 100 de clienți unici cu vârste între 18 și 53 de ani, înălțimi aproximativ între 153 și 187 cm și reprezentare echilibrată a ambelor genuri conform configurării de bază.

Fluctuația greutății fiecărui client sintetic a fost calculată conform relației:

$$\Delta g = \frac{\Delta_{\text{caloric}}}{7700} \times \Delta_{\text{zile}} \quad (3.4)$$

unde Δ_{caloric} reprezintă diferența dintre calorii consumate și TDEE, iar Δ_{zile} numărul de zile scurse de la ultima măsurătoare. Formula se bazează pe principiul larg acceptat în literatura nutrițională conform căruia 7.700 kcal sunt per kilogram de țesut adipos, valoare derivată din regula clasică a lui Wishnofsky de 3.500 kcal per pound [16].

Aderența zilnică la planul alimentar a fost simulată printr-o distribuție normală cu media 72% și abatere standard de 4%, limitată între 30% și 100% pentru a reflecta variația unui comportament real. În plus, a fost adăugat un zgomot biologic $\mathcal{N}(0, 0, 02)$ la fiecare măsurătoare de greutate, simulând fluctuații naturale ale greutății corporale independente de aportul caloric, precum variațiile de hidratare sau retenția de apă.

Structura înregistrărilor sintetice urmează același vector de caracteristici utilizat la predicție, asigurând consistența între antrenare și inferența pe date reale. Procesul de generare produce între 3.600 și 6.000 înregistrări de greutate (100 clienți x 6 luni x 6/10 măsurători/lună). Din acestea, prin aplicarea unei ferestre glisante de 30 de zile care necesită minimum patru măsurători și cel puțin o înregistrare viitoare pentru a calcula ritmul real de modificare a greutății, rezultă între aproximativ 3.200 și 5.600 exemple de antrenare.

3.4.1.2 Modelul YOLOv8

Setul de date constă în 18.555 de imagini repartizate în 45 de clase, construit în platforma Roboflow care pune la dispoziție instrumente pentru organizarea, etichetarea și augmentarea imaginilor. Clasele fac parte din diverse categorii alimentare precum fructe, legume, carne, lactate și paste, iar imaginile au fost obținute prin concatenarea mai multor seturi de date găsite pe platformă. Acestea aveau deja aplicate tehnici de preprocesare precum redimensionare la 640x640 pixeli cu margini negre și tehnici de augmentare: flip orizontal, rotație între -15° și $+15^\circ$, luminozitate între -25% și $+25\%$. Fiecare imagine

conține în medie trei adnotări, totalul bounding box-urilor fiind de 49.824. Setul de date a fost împărțit în 80% antrenare, 10% validare și 10% testare.

Antrenarea s-a realizat în Google Colab datorită performanței GPU ridicate, modelul fiind evaluat la finalul fiecărei epoci pe subsetul de validare. Acesta a fost antrenat pe 50 de epoci, fără ca mecanismul de *early stopping* (`patience=10`) să se activeze, semn că performanța a continuat să se îmbunătățească până la final. S-a utilizat dimensiunea batch de 16, optimizatorul *AdamW*, rata de învățare de 0,001, factorul final al ratei de învățare de 0,01 și regularizare L2. Justificările pentru alegerea acestor hiperparametri se regăsesc în Anexa C.

3.4.2 Metodologia de testare

Modelul Random Forest Regressor a fost evaluat utilizând cross-validation, datele fiind împărțite în cinci părți egale unde acesta se antrenează pe patru și se testează pe cea rămasă, procesul repetându-se de cinci ori. Metrica rezultată este coeficientul R^2 , care evaluează cât de bine este estimat ritmul săptămânal de modificare a greutatei. O valoare apropiată de 1 indică predicții precise, iar valori apropiate de 0 sau negative indică performanțe reduse. Funcționalitatea a fost testată pe clienți reali cu obiective de deficit și surplus caloric, validându-se comportamentul corect în ambele situații. Mecanismul de fallback, activat la date insuficiente, afișează un mesaj corespunzător care îndeamnă utilizatorul să înregistreze cel puțin patru măsurători de greutate pentru a accesa predicția.

Modelul YOLOv8 a fost evaluat pe subsetul de testare, reprezentând 10% din totalul imaginilor. Testarea a urmărit cât de bine reușește acesta să detecteze și localizeze corect alimente în fotografii noi, prin compararea predicțiilor cu adnotările reale. Varianta selectată pentru testare a fost cea care a obținut cel mai bun scor mAP50-95 pe subsetul de validare în cursul antrenării. Performanța a fost observată pe baza metricilor clasice, precum precision, recall și mAP, care indică capacitatea de localizare și clasificare a claselor.

Restul funcționalităților au fost testate manual prin scenarii specifice fiecărui rol, cu atenție asupra cazurilor limită predispuse la erori. Evaluarea a urmărit comportamentul corect al componentelor individuale în cadrul unor fluxuri complete de utilizare. Exemple de scenarii testate: clientul utilizează fluxul pentru sugestii de mese, iar cererea este transmisă printr-o notificare nutriționistului, specialistul construiește și publică un plan alimentar, iar clientul primește o notificare care îl redirecționează corect către planul publicat. Programările au necesitat o atenție deosebită deoarece implică un număr mare de stări și tranziții. Scenariile testate au acoperit toate combinațiile posibile: clientul creează sau anulează o programare, nutriționistul o confirmă sau o respinge, iar notificările corespunzătoare sunt livrate corect fiecărui rol în parte.

3.4.3 Rezultate

3.4.3.1 Rezultatele modelului Random Forest Regressor

Media scorurilor R^2 este $0,942 \pm 0,012$, indicând o capacitate ridicată de generalizare. Suplimentar, scorul Out-of-Bag de 0,967 confirmă că modelul nu este supraantrenat pe datele sintetice. Această metrică de evaluare internă este specifică algoritmului Random Forest, utilizându-se eşantioane care nu au fost incluse în antrenarea anumitor arbori.

Analiza importanței caracteristicilor a scos în evidență că numărul de kilograme rămase până la obiectiv domină predicția cu o pondere de 62,86% deoarece este direct legată de numărul estimat de săptămâni necesare atingerii acestuia. Celelalte variabile, precum aderența la planul nutrițional și deficitul/surplusul caloric față de TDEE, cu ponderi de 17,12%, respectiv 16,12% contribuie la ajustarea vitezei estimate de progres. Restul caracteristicilor au contribuții individuale sub 1%.

3.4.3.2 Rezultatele modelului YOLOv8

Modelul YOLOv8 a fost evaluat în final pe subsetul de test, format din 1.855 imagini și 4.982 instanțe distribuite între cele 45 de clase. Rezultatele globale au fost: mAP50=86,6%, mAP50-95=70,2%, Precision=85,5% și Recall=81,6%. Primele două metrice se diferențiază prin nivelul de exigență: mAP50 consideră o corecție validă dacă zona detectată se suprapune în proporție de cel puțin 50% cu obiectul real, în timp ce mAP50-95 urmărește o suprapunere cât mai exactă a bounding box-urilor.

S-a analizat și performanța per clasă (detalii și grafice se regăsesc în Anexa D), iar cele mai bune rezultate au fost obținute de clase precum *apple* (mAP50=99,4%), *egg* (94,9%) și *potato* (92,7%) deoarece au un aspect vizual ușor de recunoscut, în timp ce clase precum *zucchini* (79,8%) și *pork* (61,7%) au înregistrat scoruri mai scăzute, fapt explicat de asemănarea vizuală cu alte clase. Din perspectiva vitezei, modelul procesează o imagine în 23,2 ms ceea ce confirmă compatibilitatea pentru utilizarea în timp real în cadrul serviciului Python.

3.5 Analiza comparativă și interpretarea rezultatelor

3.5.1 Punctele forte ale soluției implementate

BitePlan reușește să integreze tehnologii complexe (YOLOv8, Random Forest, LLM-uri, API-uri externe) într-un flux coerent și accesibil utilizatorilor. Față de o consultație clasică, aplicația centralizează funcționalități care ar necesita instrumente separate: sugestii de mese personalizate, programări, setarea obiectivului de greutate, organizarea listei de cumpărături și comunicarea directă cu specialistul.

Din perspectiva nutriționistului, aplicația adoptă soluții pentru gestionarea eficientă a activității: organizarea consultațiilor și a programului de lucru, setarea obiectivelor nutriționale, planificarea regimurilor alimentare și vizualizarea progresului clienților.

3.5.2 BitePlan în raport cu soluțiile actuale

Aplicațiile de nutriție MyFitnessPal și Noom pun accentul pe monitorizarea individuală și autodisciplină, însă studiile arată că lipsa unei ghidări personalizate conduce la întreruperea timpurie a regimului alimentar [6]. BitePlan adresează această problemă prin integrarea unui specialist care oferă suport continuu.

O altă provocare este efortul ridicat al auto-monitorizării alimentare, ceea ce afectează folosirea aplicațiilor nutriționale [1]. BitePlan adresează această provocare prin automatizarea mai multor procese: logarea meselor se realizează prin butoanele „Ate” și „Skip”, lista de cumpărături este generată automat, iar sugerarea meselor este inițiată prin scanarea unui produs preferat.

Din perspectiva detecției vizuale a alimentelor, modelul YOLOv8 integrat în BitePlan obține un mAP50 de 86,6%, superior față de 76,36% corespunzător componentei YOLOv2 adoptată în aplicația FoodTracker. În plus, se obține o viteză de inferență de 23,2 ms per imagine, superioară pragului de 75 ms raportat de aceeași platformă [13].

3.5.3 Interpretarea personală a rezultatelor

Privind retrospectiv asupra întregului proiect, consider că BitePlan se diferențiază de soluțiile existente prin echilibrul dintre autonomia clientului și supervizarea specialistului. Clienții au posibilitatea de a contribui activ la regimul alimentar, dar decizia finală este responsabilitatea nutriționistului.

Rezultatele modelelor Random Forest Regressor și YOLOv8 indică potențialul dezvoltării BitePlan într-un cadru industrial. Utilizarea aplicației de către un număr mai mare de utilizatori poate contribui la îmbunătățirea preciziei componentelor inteligente.

3.5.4 Aplicabilitate reală și potențial de dezvoltare

Din punct de vedere tehnic, BitePlan a fost construită pe o arhitectură de micro-servicii ceea ce permite îmbunătățirea independentă a fiecărei componente fără a afecta stabilitatea întregii platforme.

Din perspectiva aplicabilității reale, BitePlan poate fi utilizat într-un context real, precum o clinică de nutriție sau de către un specialist independent, oferind funcționalități complexe. Aplicația este intuitivă, interfața modernă, iar fluxurile de lucru urmăresc necesitățile ambelor roluri de la gestionarea programărilor până la planificarea meselor și monitorizarea progresului.

Capitolul 4

Concluzii și propuneri viitoare

4.1 Concluzii generale

BitePlan creează contextul unei comunicări directe între client și nutriționist, centralizând într-un singur loc necesitățile relației dintre cele două roluri, eliminând nevoia de instrumente separate.

Arhitectura pe microservicii încurajează dezvoltarea independentă a fiecărei componente, poziționând BitePlan ca o soluție viabilă atât pentru utilizare individuală, cât și pentru un context profesional real.

Deciziile de design sunt susținute de articole de specialitate, de la distribuția calorică a meselor și calculul TDEE, până la importanța unei ghidări specializate pentru adoptarea pe termen lung a unui comportament alimentar echilibrat. Interfața intuitivă, însoțită de explicații privind utilizarea fluxurilor, asigură o experiență fluidă, în timp ce opțiuni precum notificări in-app, suport dark-mode și autentificare prin Google îmbunătățesc interacțiunea zilnică a utilizatorului.

4.2 Perspective de dezvoltare

O primă direcție vizează tranziția automată a programărilor în statusul „Completed”. Soluția curentă actualizează statusul la repornirea manuală a serverului, ceea ce nu este viabil într-un scenariu real de producție. Înlocuirea acesteia cu un *cron job* (sarcină programată să ruleze la intervale fixe de timp) ar asigura actualizarea continuă.

Automatizarea mecanismului de fine-tuning al modelului YOLOv8 reprezintă o altă direcție importantă. Atingerea unui prag minim de corecții ar putea declanșa automat reantrenarea modelului și înlocuirea acestuia în microserviciu.

În ceea ce privește sugestiile de mese, înlocuirea API-ului Spoonacular sau completarea acestuia cu un API dedicat produselor românești ar spori relevanța recomandărilor pentru utilizatorii din România.

Bibliografie

- [1] Juliana Chen, Jessica Lieffers, Adrian Bauman, Rhona Hanning și Margaret Allman-Farinelli, „Designing health apps to support dietetic professional practice and their patients: qualitative results from an international survey”, în *JMIR mHealth and uHealth* 5.3 (2017), e6945.
- [2] Sang Ouk Chin, Changwon Keum, Junghoon Woo, Jehwan Park, Hyung Jin Choi, Jeong-taek Woo și Sang Youl Rhee, „Successful weight reduction and maintenance by using a smartphone application in those with overweight and obesity”, în *Scientific reports* 6.1 (2016), p. 34563.
- [3] Guillermo Díaz, Mónica Pellejero, Raquel Sánchez-Sánchez și Agustín J Sánchez-Medina, „The role of trust in health care professionals in the diet app context: an approach based on the extended UTAUT model”, în *Humanities and Social Sciences Communications* 13.1 (2026), p. 126.
- [4] Mitchell Gordon, Tim Althoff și Jure Leskovec, „Goal-setting and achievement in activity tracking apps: a case study of MyFitnessPal”, în *The World Wide Web Conference*, 2019, pp. 571–582.
- [5] Sandra van der Haar, Ireen Raaijmakers, Muriel CD Verain și Saskia Meijboom, „Incorporating consumers’ needs in nutrition apps to promote and maintain use: mixed methods study”, în *JMIR mHealth and uHealth* 11.1 (2023), e39515.
- [6] Robert Jakob, Justas Narauskas, Elgar Fleisch, Laura Maria König și Tobias Kowatsch, „Factors associated with adherence to a public mobile nutritional health intervention: Retrospective cohort study”, în *Computers in Human Behavior Reports* 15 (2024), p. 100445.
- [7] Rachael Jinnette, Ai Narita, Byron Manning, Sarah A McNaughton, John C Mathers și Katherine M Livingstone, „Does personalized nutrition advice improve dietary intake in healthy adults? A systematic review of randomized controlled trials”, în *Advances in Nutrition* 12.3 (2021), pp. 657–669.
- [8] Yi-Chin Kato-Lin, Vibhanshu Abhishek, Julie S Downs și Rema Padman, „Food for thought: The impact of m-health enabled interventions on eating behavior”, în *Available at SSRN* 2736792 (2016).

- [9] Meelim Kim, Youngin Kim, Yoonjeong Go, Seokoh Lee, Myeongjin Na, Younghee Lee, Sungwon Choi și Hyung Jin Choi, „Multidimensional cognitive behavioral therapy for obesity applied by psychologists using a digital platform: open-label randomized controlled trial”, în *JMIR mHealth and uHealth* 8.4 (2020), e14817.
- [10] Simon Mezgec și Barbara Koroušić Seljak, „NutriNet: a deep learning food and drink image recognition system for dietary assessment”, în *Nutrients* 9.7 (2017), p. 657.
- [11] Mark D Mifflin, Sachiko T St Jeor, Lisa A Hill, Barbara J Scott, Sandra A Daugherty și Young O Koh, „A new predictive equation for resting energy expenditure in healthy individuals”, în *The American journal of clinical nutrition* 51.2 (1990), pp. 241–247.
- [12] G Roman, C Bala, G Creteanu, M Graur, M Morosanu, P Amorin, L Pîrcalaboiu, G Radulian, R Timar și A Achimas Cadariu, „OBESITY AND HEALTH-RELATED LIFESTYLE FACTORS IN THE GENERAL POPULATION IN ROMANIA: A CROSS SECTIONAL STUDY.”, în *Acta Endocrinologica (1841-0987)* 11.1 (2015).
- [13] Jianing Sun, Katarzyna Radecka și Zeljko Zilic, „Foodtracker: A real-time food detection mobile application by deep convolutional neural networks”, în *arXiv preprint arXiv:1909.05994* (2019).
- [14] Gabrielle M Turner-McGrievy, Caroline Glagola Dunn, Sara Wilcox, Alycia K Boutté, Brent Hutto, Adam Hoover și Eric Muth, „Defining adherence to mobile dietary self-monitoring and assessing tracking over time: tracking at least two eating occasions per day is best marker of adherence within two different mobile health randomized weight loss interventions”, în *Journal of the Academy of Nutrition and Dietetics* 119.9 (2019), pp. 1516–1524.
- [15] Lelia Voinea, Diana Maria Vrânceanu, Alina Filip, Dorin Vicențiu Popescu, Teodor Mihai Negrea și Răzvan Dina, „Research on food behavior in Romania from the perspective of supporting healthy eating habits”, în *Sustainability* 11.19 (2019), p. 5255.
- [16] Max Wishnofsky, „Caloric equivalents of gained or lost weight”, în *Journal of the American Medical Association* 173.1 (1960), pp. 85–85.

Anexa A

Implementarea arhitecturii MVVM și comunicarea cu backend-ul

Arhitectura MVVM (Model-View-ViewModel) este organizată în trei straturi cu responsabilități distincte, fiecare urmând să fie prezentat în detaliu.

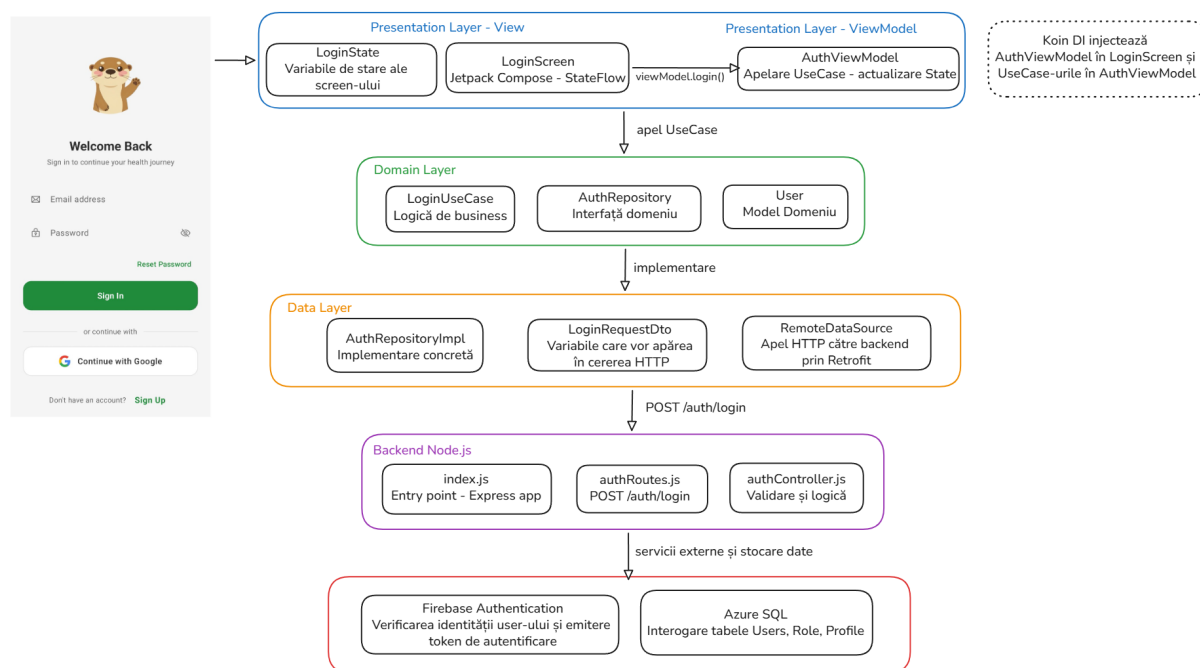


Figura A.1: Implementarea fluxului de autentificare utilizând arhitectura MVVM, de la interfața utilizatorului până la comunicarea cu backend-ul Node.js și serviciile externe

Stratul de prezentare (Presentation Layer) este componenta aplicației care interacționează cu utilizatorul. Este alcătuit din două module: View-ul implementat cu framework-ul declarativ Jetpack Compose, care reacționează automat la modificările de stare prin StateFlow și ViewModel, care deține starea ecranului și organizează apeluri către logica de business prin UseCase-uri. Rolul acestui strat este să se ocupe de interfața grafică și afișarea datelor, nu de ceea ce semnifică acestea.

Stratul de domeniu (Domain Layer) reprezintă „creierul” aplicației Android deoarece conține logica de business și regulile care dictează cum se comportă platforma. Este alcătuit din UseCase-uri care definesc operațiile disponibile în aplicație, entități corespunzătoare datelor aplicației și Repository-uri care reprezintă interfețe, rolul lor fiind acela de a separa logica de business de detaliile de implementare ale sursei de date.

Stratul de date (Data Layer) este responsabil pentru accesul la surse externe, în cazul de față backend-ul Node.js. Acesta conține implementările concrete ale Repository-urilor, DTO-uri (Data Transfer Objects) care modelează structura datelor schimbate prin HTTP și RemoteDataSource, care gestionează apelurile efective către backend prin biblioteca Retrofit.

Comunicarea Android - Backend este realizată prin Retrofit, care serializează și deserializează automat JSON-ul prin convertorul Gson. Fiecare endpoint al backend-ului Node.js este reprezentat printr-o funcție adnotată (@GET, @POST, @PUT, @PATCH, @DELETE) în interfața Kotlin, iar Retrofit generează automat la runtime implementarea care construiește apelul HTTP corespunzător.

Comunicarea Backend - servicii externe se realizează prin apeluri HTTP către Firebase Authentication pentru validarea token-urilor, Azure SQL pentru accesul și stocarea datelor relaționale și API-uri externe precum Spoonacular, Open Food Facts, DeepL și Groq, fiecare utilizat în funcționalități complexe.

Anexa B

Vectorul de caracteristici și fluxul de predicție

Caracteristicile utilizate au fost grupate în patru categorii principale: greutate, aderență, aport caloric și profilul utilizatorului.

Caracteristicile legate de greutate includ: prima greutate înregistrată în fereastra de 30 de zile, ultima greutate înregistrată, tendința greutății (slăbire/îngrășare), abaterea standard a greutăților, care reflectă variabilitatea măsurărilor, diferența simplă între ultima și prima greutate din fereastră, precum și numărul de kilograme rămase până la atingerea obiectivului țintă.

Caracteristicile legate de aderență includ: aderența medie la plan, precum și abaterea standard a aderenței, care arată consistența comportamentului alimentar.

Caracteristicile legate de aportul caloric includ: media kaloriilor consumate efectiv pe zi din fereastră, obiectivul caloric zilnic stabilit de nutriționist, care este constant per fereastră, precum și diferența medie dintre kaloriile consumate efectiv și TDEE, conform ecuației 3.3, care arată deficitul sau surplusul caloric.

Caracteristicile legate de profilul utilizatorului includ: nivelul de activitate fizică transformat numeric: sedentar (0,00), ușor activ (0,33), activ (0,67), foarte activ (1,00), genul în format binar: masculin (1), feminin (0), vârsta în ani și BMR calculat utilizând formulele prezentate în ecuațiile 3.1 și 3.2. Este important de menționat că valorile numerice asociate nivelului de activitate fizică au fost alese prin normalizarea în intervalul [0,1] pentru a facilita procesarea eficientă a datelor de către modelul de Machine Learning.

Fluxul complet de la vectorul de caracteristici descris anterior la răspunsul final transmis aplicației Android este prezentat în Figura B.1.

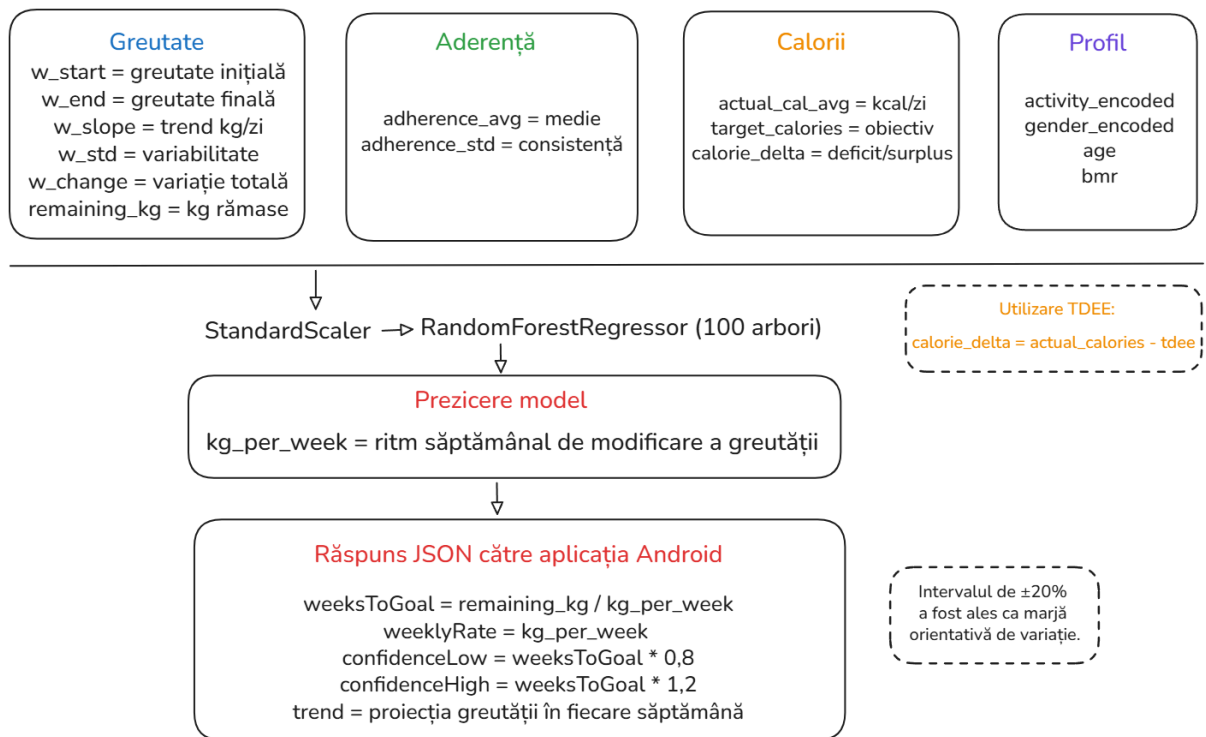


Figura B.1: Fluxul predicției de la vectorul de caracteristici până la răspunsul JSON

Anexa C

Motivarea alegerii hiperparametrilor pentru modelul YOLOv8

```
results = model.train(  
    data="/content/Food-Detection-Thesis-6/data.yaml",  
    epochs=50,  
    patience=10,  
    imgsz=640,  
    batch=16,  
    optimizer="AdamW",  
    lr0=0.001,  
    lrf=0.01,  
    weight_decay=0.0005,  
    warmup_epochs=5,  
    save=True,  
    save_period=7,  
    plots=True,  
    val=True,  
    device=0,  
    workers=2,  
    project="/content/drive/MyDrive/LICENTA/food_detection_v6",  
    name="yolov8m_v6",  
    exist_ok=True,  
)
```

Figura C.1: Captură de ecran extrasă din Google Colab

Pentru vizualizarea clară a hiperparametrilor utilizați la antrenarea modelului se poate vizualiza Figura C.1.

Numărul de epoci (`epochs=50`) a fost ales optim astfel încât să existe suficient timp de antrenare. Inițial a fost încercată varianta cu 30 de epoci, dar performanța pe subsetul de evaluare a metricii mAP50-95 (67,3%) și mAP-50(84,8%) au putut fi îmbunătățite până la 70%, respectiv 86,1%.

Valoarea pentru **oprirea anticipată** (`patience=10`) a permis modelului mai mult timp pentru a recupera performanța, dar fără a risca supraantrenarea. O valoare mai mică ar fi putut împiedica îmbunătățirea modelului.

Dimensiunea imaginilor (`imgsz=640`) reprezintă rezoluția standard YOLOv8, oferind un echilibru între acuratețe și viteză. O rezoluție mai mare (de exemplu 1280) ar fi crescut drastic timpul de antrenare și memoria GPU fără rezultate considerabile.

Dimensiunea batch-ului (`batch=16`) a fost aleasă în funcție de memoria GPU disponibilă în Google Colab, asigurând estimări stabile ale gradientului fără a depăși limita VRAM (Video RAM).

Algoritmul folosit pentru **actualizarea parametrilor** modelului (`optimizer="AdamW"`) adoptă o variantă îmbunătățită a versiunii *Adam* care separă `weight_decay` de actualizarea gradientului, reducând tendința de supraantrenare.

Valoarea inițială pentru rata de învățare (`lr0=0,001`) este standard pentru *AdamW* în transfer learning. O valoare mai mare (de exemplu 0,01) ar fi destabilizat greutatea pre-antrenate, respectiv mai mică (de exemplu 0,0001) ar fi încetinit excesiv convergența.

Factorul final al ratei de învățare (`lrf=0,01`) definește valoarea la care rata de învățare scade progresiv pe parcursul antrenării, conform formulei $lr_{final} = lr_0 \times lrf = 0,001 \times 0,01 = 0,00001$. Reducerea treptată spre final permite ajustări fine ale greutăților în loc de salturi mari, favorizând stabilitatea convergenței.

Mecanismul de regularizare L2 (`weight_decay=0,0005`) este standard pentru Computer Vision. Acesta penalizează greutățile mari și reduce riscul de supraantrenare pe un set de date de dimensiune medie.

Hiperparametrul `warmup_epochs=5` a fost introdus deoarece rata de învățare crește gradual de la 0 la `lr0` în primele cinci epoci. În cazul neutilizării, actualizările bruște din primele epoci ar fi putut deteriora greutățile deja învățate de modelul pre-antrenat.

Hiperparametrul `save_period=7` a fost adoptat ca măsură de precauție. Permite reluarea antrenării de la ultimul checkpoint salvat în cazul expirării sesiunii Colab.

Hiperparametrul `device=0` reprezintă utilizarea primului GPU disponibil în loc de CPU pentru a reduce timpul de antrenare per epocă.

Hiperparametrul `workers=2` semnifică numărul de procese pentru încărcarea datelor deoarece Google Colab are memorie partajată limitată între procese. O valoare prea mare (de exemplu 8) ar fi cauzat erori de tip *out-of-shared-memory* în timpul antrenării.

Restul hiperparametrilor care nu au fost menționați (`data`, `project`, `name`, `exist_ok`) sunt utilizați pentru încărcarea setului de date, salvarea modelului într-un drive dedicat și generarea unor grafice.

Anexa D

Grafice generate de testarea modelului YOLOv8

Matricea de confuzie normalizată din **Figura D.1** este un instrument de evaluare a performanței per clasă, fiecare rând semnifică clasa prezisă de model, iar fiecare coloană clasa reală din setul de test. Diagonala principală urmărește proporția instanțelor clasificate corect, 1,0 însemnând o acuratețe ridicată. Restul valorilor semnifică confuzii între clase, celula este mai saturată cu cât confuzia este mai frecventă.

Matricea confirmă că modelul clasifică corect marea majoritate a claselor, diagonala principală fiind predominant saturată. Clase precum *cheese*, *butter*, *pork*, *garlic* și *chicken* prezintă erori din cauza ambiguității vizuale ridicate precum și confuzia cu fundalul, aspect specific seturilor de date care detectează alimente utilizând imagini reale.

Curba F1-Confidence ilustrată în **Figura D.2** prezintă echilibrul dintre numărul de detecții corecte (Precision) și câte obiecte reale a găsit modelul (Recall). Rezultatul **F1=0,83 la un confidence threshold de 0,452** semnifică faptul că pragul optim de detecție este de aproximativ 45%. Pragurile mai mici cresc numărul de detecții false pozitive, iar pragurile mai mari devin prea selective, determinând omiterea alimentelor reale din imagine. Rezultatul de 0,83 este considerat potrivit pentru un dataset cu 45 de clase, unde multe produse alimentare prezintă similarități vizuale.

Curba Precision-Recall disponibilă la **Figura D.3** ilustrează performanța reală a modelului indiferent de pragul de încredere ales. O curbă ideală ar rămâne la Precision = 1,0 indiferent de câte obiecte găsește, însă cu cât modelul încearcă să găsească mai multe alimente, cu atât crește riscul de detecții greșite. Aria de sub curbă reprezintă scorul **mAP50=0,866**, confirmând că modelul menține un echilibru optim între a detecta corect alimente și a nu le omite în imagini.

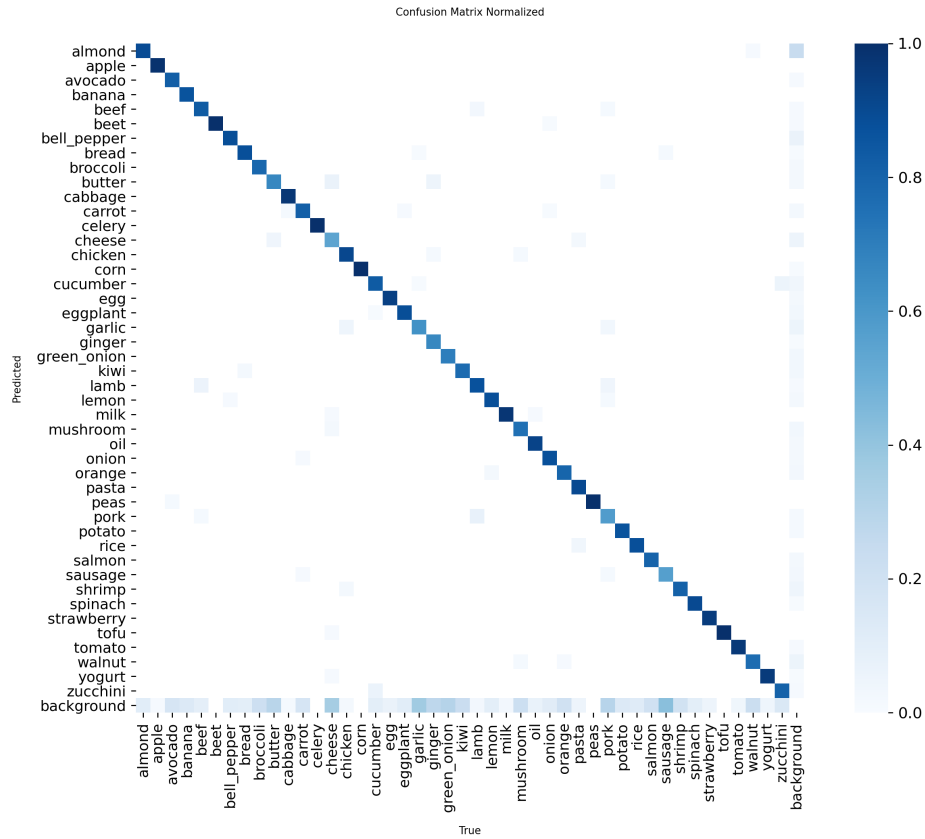


Figura D.1: Matricea de confuzie normalizată

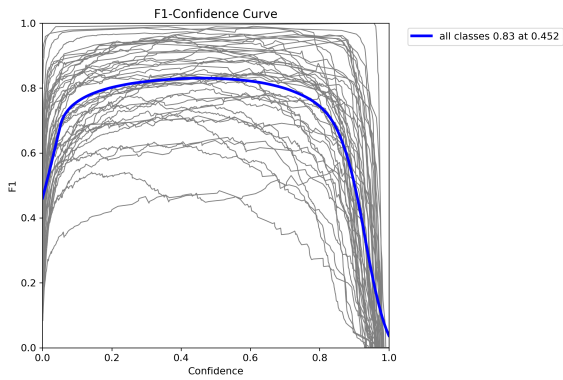


Figura D.2: Curba F1-confidence

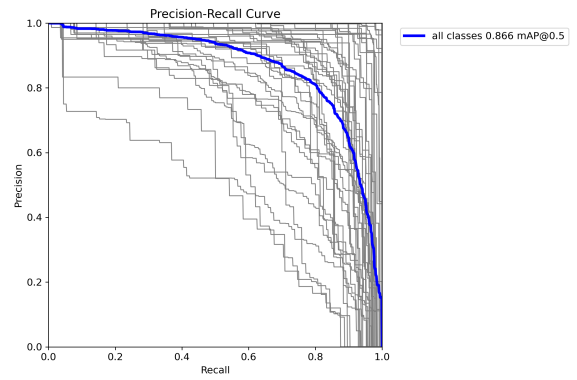


Figura D.3: Curba PR (Precision-Recall)